

JAVA CODING INTERVIEW CHEATSHEET

Optimized Compact Reference for Common Array, String, and Dynamic Programming Problems

1. Find Second Largest/Smallest

```
int largest = Integer.MIN_VALUE, secondLargest = Integer.MIN_VALUE;
for (int num : arr) {
    if (num > largest) { secondLargest = largest;
    largest = num; }
    else if (num > secondLargest && num != largest)
    { secondLargest = num; }
}
```

2. Reverse Array / String

```
int left = 0, right = arr.length - 1;
while (left < right) {
    int temp = arr[left]; arr[left] = arr[right];
    arr[right] = temp;
    left++; right--;
}
```

3. Count Even And Odd Numbers

```
int even = 0, odd = 0;
for (int num : arr) {
    if (num % 2 == 0) even++; else odd++;
}
// Stream Approach
List evens = Arrays.stream(nums).boxed().filter(n -> n % 2 == 0).collect(Collectors.toList());
List odds = Arrays.stream(nums).boxed().filter(n -> n % 2 != 0).collect(Collectors.toList());
```

4. Remove Duplicates From Array

```
int[] unique =
Arrays.stream(arr).distinct().toArray();
```

5. Find Duplicate Elements

```
Set seen = new HashSet<>(), duplicates = new
LinkedHashSet<>();
for (int num : arr) {
    if (!seen.add(num)) duplicates.add(num);
}
```

6. Frequency of Elements/Chars

```
Map map = new LinkedHashMap<>();
for (int num : arr) map.put(num,
map.getOrDefault(num, 0) + 1);
```

7. Move Zeros To End

```
int nonZeroPos = 0;
for (int i = 0; i < nums.length; i++) {
    if (nums[i] != 0) {
        int temp = nums[i]; nums[i] =
nums[nonZeroPos]; nums[nonZeroPos] = temp;
        nonZeroPos++;
    }
}
```

8. Merge Two Arrays

```
int[] merged = IntStream.concat(
    Arrays.stream(arr1 == null ? new int[0] :
arr1),
    Arrays.stream(arr2 == null ? new int[0] : arr2)
).toArray();
```

9. Intersection of Two Arrays

```
Set set =
Arrays.stream(arr1).boxed().collect(Collectors.toSe
t());
int[] intersect =
Arrays.stream(arr2).distinct().filter(num ->
set.contains(num)).toArray();
```

10. Two Sum Problem

```
List result = new ArrayList<>();
for (int i = 0; i < nums.length; i++) {
    for (int j = i + 1; j < nums.length; j++) {
        if (nums[i] + nums[j] == target)
            result.add(new int[]{i, j});
    }
}
```

11. Find Combination of Subsets

```
public static void findCombinations(int[] nums, int
target) {
    backtrack(nums, target, 0, new ArrayList<>());
}
private static void backtrack(int[] nums, int
target, int start, List current) {
    if (target == 0) { System.out.println(current);
return; }
    if (target < 0) return;
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrack(nums, target - nums[i], i + 1,
current);
        current.remove(current.size() - 1);
    }
}
```

12. Max Subarray Sum (Kadane's)

```
int currentSum = arr[0], maxSum = arr[0], start =
0, end = 0, tempStart = 0;
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > currentSum + arr[i]) { currentSum
= arr[i]; tempStart = i; }
    else { currentSum += arr[i]; }
    if (currentSum > maxSum) { maxSum = currentSum;
start = tempStart; end = i; }
}
int[] sub = Arrays.copyOfRange(arr, start, end +
1);
```

13. Best Time to Buy/Sell Stock

```
int minPrice = prices[0], buyDay = 0, sellDay = 0,
maxProfit = 0;
for (int i = 1; i < prices.length; i++) {
    if (prices[i] < minPrice) { minPrice =
prices[i]; buyDay = i; }
    if (prices[i] - minPrice > maxProfit)
{ maxProfit = prices[i] - minPrice; sellDay = i; }
}
```

14. First Non-Repeating / Most Repeating

```
Map map = new LinkedHashMap<>();
for (int num : arr) map.put(num,
map.getDefault(num, 0) + 1);
Integer firstNonRepeating = null, mostRepeating =
null; int maxCount = 0;
for (Map.Entry entry : map.entrySet()) {
    if (firstNonRepeating == null &&
entry.getValue() == 1) firstNonRepeating =
entry.getKey();
    if (entry.getValue() > maxCount) { maxCount =
entry.getValue(); mostRepeating = entry.getKey(); }
}
```

15. Sort Ascending/Descending Array

```
int[] asc = Arrays.stream(arr).sorted().toArray();
int[] desc =
Arrays.stream(arr).boxed().sorted(Collections.rever
seOrder()).mapToInt(Integer::intValue).toArray();
```

16. Palindrome Check

```
int left = 0, right = str.length() - 1;
while (left < right) {
    if (str.charAt(left) != str.charAt(right))
return false;
    left++; right--;
}
return true;
```

17. Anagram Check

```
if (s1.length() != s2.length()) return false;
char[] a = s1.toCharArray(), b = s2.toCharArray();
Arrays.sort(a); Arrays.sort(b);
return Arrays.equals(a, b);
```

18. Count Occurrence of Character

```
int count = 0;
for (char ch : str.toCharArray()) { if (ch ==
target) count++; }
```

19. Remove Duplicate Characters

```
Set set = new LinkedHashSet<>();
for (char ch : str.toCharArray()) set.add(ch);
StringBuilder sb = new StringBuilder();
for (char ch : set) sb.append(ch);
String result = sb.toString();
```

20. Reverse Words in Sentence

```
String[] words = str.split("\\s+");
StringBuilder result = new StringBuilder();
for (int i = words.length - 1; i >= 0; i--)
result.append(words[i]).append(" ");
String finalStr = result.toString().trim();
```

21. Reverse Each Word in Sentence

```
String[] words = sentence.split(" ");
StringBuilder result = new StringBuilder();
for (String word : words)
result.append(reverseString(word)).append(" ");
```

22. Longest Common Prefix

```
String prefix = strs[0];
for (int i = 1; i < strs.length; i++) {
    if (strs[i] == null) return "";
    while (!strs[i].startsWith(prefix)) {
        prefix = prefix.substring(0,
prefix.length() - 1);
        if (prefix.isEmpty()) return "";
    }
}
```

23. Longest Common Subsequence

```
int m = s1.length(), n = s2.length();
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i - 1) == s2.charAt(j - 1))
dp[i][j] = 1 + dp[i - 1][j - 1];
        else dp[i][j] = Math.max(dp[i - 1][j],
dp[i][j - 1]);
    }
}
return dp[m][n];
```

24. Balance Valid Parentheses

```
public static boolean isValid(String str) {
    Stack stack = new Stack<>();
    for (char ch : str.toCharArray()) {
        if (ch == '(' || ch == '{' || ch == '[')
{ stack.push(ch); }
        else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((ch == ')' && top != '(') || (ch ==
'}' && top != '{') || (ch == ']' && top != '['))
return false;
        }
    }
    return stack.isEmpty();
}
```

25. Strings Equal Without .equals()

```
if (s1.length() != s2.length()) return false;
for (int i = 0; i < s1.length(); i++) {
    if (s1.charAt(i) != s2.charAt(i)) return false;
}
return true;
```